
GENOME-FACTORY: A Library for Tuning, Deploying, and Interpreting Genomic Foundation Models

Weimin Wu^{*1} Xuefeng Song^{*1} Yibo Wen^{*1}
Qinjie Lin¹ Zhihan Zhou¹ Jerry Yao-Chieh Hu¹ Zhong Wang^{2,3,4} Han Liu^{1,5}

Abstract

We introduce GENOME-FACTORY, the first integrated Python library for tuning, deploying, and interpreting genomic foundation models. Our core contribution is to simplify and unify the workflow for genomic model development: data collection, model tuning, inference, benchmarking, and interpretability. For data collection, GENOME-FACTORY offers an automated pipeline to download genomic sequences and preprocess them. For model tuning, GENOME-FACTORY supports both full and parameter-efficient fine-tuning across diverse genomic models. For inference, GENOME-FACTORY enables both embedding extraction and DNA sequence generation. For benchmarking, we include two existing benchmarks and provide a flexible interface to incorporate additional benchmarks. For interpretability, GENOME-FACTORY introduces an open-source biological interpreter based on a sparse auto-encoder. We validate the utility of GENOME-FACTORY across three dimensions: (i) Compatibility with diverse models and fine-tuning methods; (ii) Benchmarking downstream performance using two open-source benchmarks; (iii) Biological interpretation of learned representations with DNABERT-2. These results highlight its practical value for real-world genomic analysis. **GitHub:** https://github.com/WeiminWu2000/Genome_Factory.

^{*}Equal contribution ¹Center for Foundation Models and Generative AI & Department of Computer Science, Northwestern University, USA ²School of Natural Sciences, University of California at Merced, USA ³Department of Energy Joint Genome Institute, Lawrence Berkeley National Laboratory, USA ⁴Environmental Genomics and Systems Biology Division, Lawrence Berkeley National Laboratory, USA ⁵Department of Statistics and Data Science, Northwestern University, USA. Correspondence to: Weimin Wu <wwm@u.northwestern.edu>, Han Liu <hanliu@northwestern.edu>.

1 Introduction

We introduce GENOME-FACTORY, the first integrated Python library for tuning, deploying, and interpreting genomic foundation models (GFMs). GFMs have transformed biological research by enabling accurate and scalable solutions to key tasks, including epigenetic prediction (Gao et al., 2024), regulatory element discovery (Hwang et al., 2024), and species classification (Zhou et al., 2025b). These models learn from large-scale genomic data and support progress in personalized medicine, evolutionary biology, and functional genomics (Consens et al., 2025). Despite the potential of GFMs, their adoption in life sciences remains limited due to a fundamental gap between domain expertise and technical implementation. On one hand, engineers handle model training and deployment but often lack a biological context. Conversely, biologists design experiments and define scientific goals but lack expertise for large models. To address this, GENOME-FACTORY offers a unified, practical platform to bridge this gap and accelerate the broader use of GFMs in life science research.

While general-purpose language model fine-tuning frameworks such as LLaMA-Factory (Zheng et al., 2024) provide integrated tools, they do not address the unique requirements of genomics. Firstly, genomic data demands specialized handling, including support for domain-specific formats like FASTA, integration with repositories such as NCBI (Geer et al., 2010) for data acquisition, and domain-specific data preprocessing. Secondly, developers have built genomic models across diverse environments with heterogeneous dependencies. This lack of standardization makes it difficult to use models within a unified framework, and even harder to ensure compatibility with tools from the language model ecosystem. Thirdly, fine-tuning genomic models differs from language objectives: rather than instruction tuning or text generation, biological tasks often involve predicting variant effects, enhancer activity, or gene expression. These tasks require custom model adaptations and biology-informed loss functions aligned with real-world genomic objectives. Fourthly, evaluation further depends on domain-specific benchmarks, such as variant detection or regulatory site classification (Zhou et al., 2024). These diverge from

the text-based benchmarks used to assess language models. Finally, biological interpretability is central to the utility of GFMs for scientists, whereas it is not a focus of existing language model fine-tuning frameworks. As a result, the GFMs field still lacks a unified, user-friendly platform to support the full pipeline for tuning and deploying models.

To address this challenge, we introduce GENOME-FACTORY, the first unified Python library for fine-tuning, deploying, and interpreting genomic models. GENOME-FACTORY features six modular components. (i) **Genome Collector**: Retrieves genomic sequences from public repositories (e.g., NCBI (Geer et al., 2010)) and applies preprocessing such as GC content normalization and N nucleotide correction. It also includes task-specific dataset builders for histone modification, enhancer, and promoter classification, with automated region extraction and labeling. (ii) **Model Loader**: Supports diverse GFMs, including discriminative models—HyenaDNA (Nguyen et al., 2023), DNABERT-2 (Zhou et al., 2024), Caduceus (Schiff et al., 2024), and Nucleotide Transformer (Dalla-Torre et al., 2025)—as well as generative models such as EVO (Nguyen et al., 2024) and GenomeOcean (Zhou et al., 2025a). (iii) **Model Trainer**: Enables full-parameter fine-tuning and parameter-efficient methods such as low-rank adaptation (LoRA) (Hu et al., 2022) and adapter tuning (He et al., 2021). It applies to both classification and regression tasks. (iv) **Inference Engine**: Facilitates both embedding extraction and sequence generation. (v) **Benchmark**: Provides two built-in, open-source genomic benchmarks and a plugin system for incorporating custom, domain-specific evaluation tasks and datasets. (vi) **Biological Interpreter**: Enhances model interpretability through a sparse auto encoder. It disentangles embeddings into sparse latent units and aligns them with genomic features via regression against external biological readouts. This provides an open-source tool to interpret the internal representations of GFMs. GENOME-FACTORY also offers user-friendly interfaces: a zero-code command-line interface (CLI) and an intuitive Gradio-based web-based user interface (WebUI) (Abid et al., 2019). These support both non-expert users and advanced developers with minimal computational and engineering effort.

In summary, we have the following three main contributions:

- We introduce GENOME-FACTORY, the first integrated Python framework to streamline the complete genomic model workflow. It integrates six components: (i) Genome Collector: comprehensive data collection and preprocessing (Section 2.1); (ii) Model Loader: broad support for diverse genomic models (Section 2.2); (iii) Model Trainer: an easy-to-use fine-tuning pipeline (Section 2.3); (iv) Inference Engine: efficient embedding extraction and sequence generation (Section 2.4); (v) Benchmark: built-in genomic benchmarks and

extensible evaluation plugins (Section 2.5); (vi) Biological Interpreter: model interpretability via sparse auto-encoder (Section 2.6).

- Beyond flexibility and ease of use, GENOME-FACTORY is the first unified framework to bring together diverse genomic models under a single interface. This enables seamless model comparison and assists users in selecting the most suitable model for a customized task. Notably, the Biological Interpreter is an open-source tool to decode the internal representations of GFMs with a sparse auto-encoder. This provides deeper biological insights into models.
- We validate the utility of GENOME-FACTORY across three dimensions: (i) Compatibility with diverse genomic foundation models and three widely used fine-tuning methods; (ii) Benchmarking downstream performance using two open-source benchmarks: Genome Understanding Evaluation (GUE) Benchmark (Zhou et al., 2024) and Genomic Benchmarks (Grešová et al., 2023); (iii) Biological interpretation of learned representations with DNABERT-2. These results clearly highlight its end-to-end usability and practical value for real-world genomic analysis.

Organization. Section 2 details the GENOME-FACTORY, including Genome Collector, Model Loader, Model Trainer, Inference Engine, Benchmark, and Biological Interpreter. Section 3 presents the results of our experiments to evaluate GENOME-FACTORY’s effectiveness. We leave related work in Appendix C, to discuss genomic foundation models and libraries for language models.

Conflict of Interest Disclosure. The authors declare no financial conflicts of interest. The evaluated models are open-source or publicly available research models, and no author is employed by a company whose proprietary model is evaluated in this study. Funding sources and institutional support are disclosed in the Acknowledgements.

2 GENOME-FACTORY Framework

GENOME-FACTORY comprises six core modules to unify the workflow for genomic models. The Genome Collector (Section 2.1) simplifies data retrieval (e.g., from NCBI (Geer et al., 2010)) and integrates preprocessing pipelines. The preprocessing includes the quality control steps, such as sequence-length filtering, GC content normalization, and correction of N nucleotide. It also supports task-specific dataset builders for histone modification, enhancer, and promoter classification, with automated region extraction, chromosome-name harmonization, and labeling. Model Loader (Section 2.2) and Model Trainer (Section 2.3) handle model loading and fine-tuning. They support both

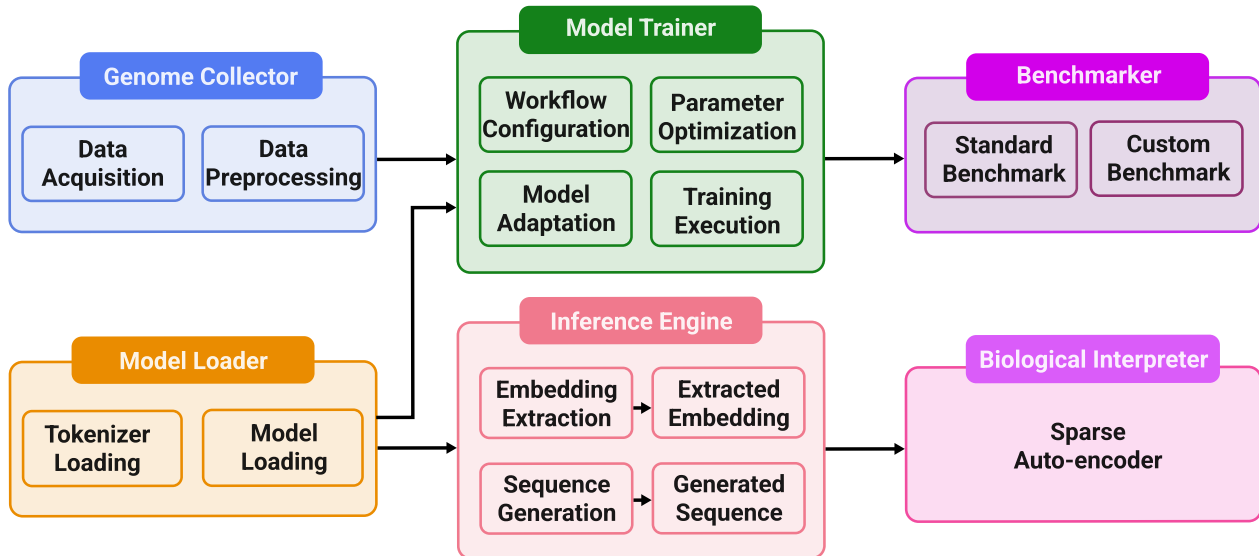


Figure 1. Overview of GENOME-FACTORY. The framework consists of six components. **Genome Collector** acquires genomic sequences from public repositories and performs preprocessing (e.g., GC normalization). **Model Loader** supports major genomic models (e.g., HyenaDNA, DNABERT-2, Caduceus, Nucleotide Transformer, EVO and GenomeOcean) and their tokenizers. **Model Trainer** configures workflows, adapts models to classification or regression tasks, and executes training with full fine-tuning or parameter-efficient methods (LoRA, adapters). **Inference Engine** enables embedding extraction and sequence generation. **Benchmark** provides standard benchmarks and allows integration of custom tasks. **Biological Interpreter** enhances interpretability through sparse auto-encoders.

full and parameter-efficient methods, such as LoRA and adapters for classification and regression tasks. The Inference Engine (Section 2.4) enables embedding extraction and sequence generation with pre-trained genomic foundation models. The Benchmark (Section 2.5) provides essential tools and datasets for rigorous model evaluation and comparison across tasks. The Biological Interpreter (Section 2.6) delivers interpretability via a sparse auto-encoder. It learns near-monosemantic latent units from model embeddings and links them to interpretable genomic features (e.g., motif presence, sequence length) through regression on external biological readouts. Users can access GENOME-FACTORY through a zero-code command-line interface or an interactive web-based user interface (Section 2.7). This design supports both flexibility and scalability for diverse applications. We provide an overview of GENOME-FACTORY in Figure 1, and **detailed usage examples** in Appendix G.

2.1 Genome Collector

The Genome Collector manages the upstream data pipeline for genomic models. It automates the fetching, transformation, and validation of genomic data. This extends beyond basic sequence downloads to include regulatory and epigenetic annotations. Standardizing dataset construction across diverse biological tasks, it ensures high-quality, task-ready inputs for the model. Furthermore, the framework supports

custom multi-stage bioinformatics pipelines.

Data Acquisition. The Genome Collector offers multiple flexible pipelines for downloading and preparing diverse DNA sequence datasets. Beyond basic genome-wide retrieval from public repositories such as NCBI, the framework supports three task-driven acquisition modes. (i) It constructs region-based datasets by identifying signal-enriched locations, such as high-coverage intervals from genome-wide profiles (e.g., histone modification enrichment). (ii) It builds binary classification datasets by separating annotated functional regions from background sequences, such as distinguishing regulatory elements from random regions (e.g., enhancers). (iii) It samples region-versus-background pairs by comparing annotated start sites to non-start regions of matched length (e.g., promoters). Each pipeline handles file downloading, genome indexing, sequence extraction, chromosome name harmonization, and sequence quality filtering. A unified interface lets users choose the task type and generate corresponding datasets with minimal effort.

Data Preprocessing. Each acquisition mode applies task-specific parsing and transformation logic. For species-level classification, the system samples fixed-length DNA fragments from genomes and assigns species labels. The histone modification pipeline extracts gene-body sequences aligned to signal peaks and binarizes them into high/low classes based on enrichment thresholds. The enhancer and promoter pipelines define positive regions from regulatory

annotations and sample negatives from non-overlapping regions. Genome Collector saves each dataset in a standardized format with DNA sequences and corresponding labels, and partitions it into training, validation, and testing sets.

Quality Control and Data Cleaning. To ensure reliable and robust downstream performance, Genome Collector applies a multi-stage quality control protocol. Initial filters enforce several basic constraints on sequence length, GC content, and the overall proportion of ambiguous nucleotides. Further statistical quality control removes outliers using three strategies: (i) filtering sequences with excessive N nucleotide; (ii) applying chi-square tests to detect compositional biases relative to expected nucleotide distributions; and (iii) removing rare compositional profiles below a predefined frequency threshold. Quality control steps confirm that the cleaned datasets exhibit balanced GC content and sequence length distributions. These safeguards ensure the final datasets have valid biological structure and robust statistical properties for training genomic models.

2.2 Model Loader

The Model Loader handles the initial phase of inference or fine-tuning by loading the genomic model and corresponding tokenizer. It leverages the HuggingFace Transformers (Wolf et al., 2019) to support diverse models, including HyenaDNA (Nguyen et al., 2023), DNABERT-2 (Zhou et al., 2024), Caduceus (Schiff et al., 2024), Nucleotide Transformer (Dalla-Torre et al., 2025), EVO (Nguyen et al., 2024) and GenomeOcean (Zhou et al., 2025a).

Tokenizer Loader. The system automatically loads the appropriate tokenizer for the selected genomic model using the Hugging Face tokenizer API. This ensures accurate and consistent encoding of input DNA sequences and full compatibility with the model’s input format.

Model Architecture and Checkpoint Loader. After initializing the tokenizer, the system loads the model with either pretrained checkpoints or random parameters. It configures the architecture based on the selected fine-tuning method. For full-parameter or LoRA fine-tuning, it loads the model and attaches a task-specific classification or regression head. For adapter fine-tuning, it freezes the base model’s weights and inserts an external adapter module for task-specific training.

2.3 Model Trainer

Adapting large genomic models to downstream tasks poses significant computational challenges due to their size and parameter count. The Model Trainer manages configuration, launches fine-tuning jobs, and monitors training. Furthermore, the framework supports the joint optimization of data preprocessing steps, such as GC normalization, together

with model training, and also supports multi-task learning. The module includes four components: workflow configuration, parameter optimization, model adaptation, and training execution. Table 1 summarizes the training strategies and system-level optimizations.

Table 1. Featured tuning techniques.

	Full	LoRA	Adapter
Mixed precision	✓	✓	✓
Flash attention	✓	✓	✓
Gradient accumulation	✓	✓	✓

Workflow Configuration. The system automatically reads user inputs and builds task-specific configurations for a classification or regression task. Custom parsers validate hyperparameters and construct training pipelines based on the selected fine-tuning strategy. The Hugging Face Trainer manages core training and distributed execution to ensure reliability, consistency, and scalability.

Parameter Optimization. GENOME-FACTORY offers three fine-tuning strategies to effectively balance resource efficiency and model adaptability: (i) Full-parameter fine-tuning updates all model parameters. It maximizes task adaptation but incurs a high computational cost. (ii) Low-rank adaptation (LoRA) (Hu et al., 2022) freezes the base model and introduces trainable low-rank matrices in attention or feed-forward layers. It reduces memory and training time while preserving performance. (iii) Adapter tuning (He et al., 2021) adds a lightweight neural module (e.g., multilayer perceptrons (Popescu et al., 2009) or convolutional neural network (O’shea and Nash, 2015)) after the frozen base model. It only updates the adapter parameters. All methods support customizable hyperparameters, e.g., learning rate, weight decay, LoRA rank, and scaling factor.

Model Adaptation. GENOME-FACTORY adjusts model architectures based on the task. For classification, it appends activation functions such as Softmax to the output layer and applies cross-entropy loss (De Boer et al., 2005). For regression, it sets up continuous-valued outputs and uses mean squared error loss (Schluchter, 2005). The system handles variable-length sequences using dynamic padding or truncation to maintain compatibility with input requirements.

Training Execution. GENOME-FACTORY seamlessly integrates the following performance optimizations: (i) Precision control supports full, FP16, and BF16 formats to reduce memory usage and speed up training on compatible hardware. (ii) Flash attention (Dao et al., 2022) accelerates attention computation and minimizes memory by avoiding explicit intermediate matrices. (iii) Gradient accumulation simulates large batch sizes, and learning rate scheduling improves convergence stability. It also applies gradient

clipping to prevent exploding gradients. Furthermore, it supports multi-GPU training via distributed data parallel (Li et al., 2020). During training, it logs evaluation metrics at configurable intervals. For classification tasks, it tracks accuracy, F1 score, precision, recall, and Matthews correlation coefficient. For regression, it records mean squared error and mean absolute error. The system saves checkpoints and retains the best-performing one based on validation metrics. This enables flexible training resumption.

2.4 Inference Engine

The Inference Engine offers a unified interface for applying genomic foundation models to two key tasks: sequence embedding extraction and DNA sequence generation.

Embedding Extraction. This component processes input DNA sequences, runs the model in evaluation mode, and extracts the final hidden state as the sequence embedding. These embeddings support downstream tasks such as classification, regression, clustering, and visualization.

Sequence Generation. This component enables compatible models to flexibly generate diverse novel DNA sequences from user-provided prompts. It supports applications such as in silico sequence variation, synthetic data augmentation, and functional sequence exploration.

2.5 Benchmarker

The Benchmarker module provides essential tools for evaluating genomic foundation models on both classification and regression tasks. It supports standardized benchmark integration, flexible plugins for custom domain-specific evaluation tasks, and automated performance evaluation.

Incorporating Benchmarks. GENOME-FACTORY includes two benchmark suites: the Genome Understanding Evaluation (GUE) Benchmark (Zhou et al., 2024) and Genomic Benchmarks (Grešová et al., 2023). It also supports plugins for integrating custom, domain-specific evaluation tasks. To use a custom benchmark, users format their data according to the Model Trainer’s input schema. For classification, the dataset should include three CSV files (training, validation, and testing). Each contains two columns: one for DNA sequences and one for integer labels. For regression, users follow the same structure but replace the label column with continuous values.

Evaluating Models. GENOME-FACTORY runs the selected model on the benchmark dataset and records detailed task-specific metrics. For classification, it computes accuracy, F1 score, precision, recall, and Matthews correlation coefficient. For regression, it computes mean squared error and mean absolute error. The system logs all results in a structured JSON file. This allows users to easily compare model performance

across tasks, datasets, or different training strategies.

2.6 Biological Interpreter

The Biological Interpreter module enables meaningful interpretation of genomic foundation models by explicitly linking internal model representations to biological features. This supports novel hypothesis generation and provides deeper insight into what the model has actually learned. However, it should be viewed as an initial step toward mechanistic interpretability for GFM’s using sparse autoencoders, rather than a fully validated tool for biological discovery.

Sparse Auto-encoder. GENOME-FACTORY uses a sparse auto-encoder to disentangle latent embeddings from genomic models. The workflow involves three stages: (i) Sequence embedding extraction: The system embeds input DNA sequences with a pretrained genomic model, such as GenomeOcean. (ii) Sparse auto-encoder training: It trains a sparse auto-encoder on these embeddings with a reconstruction loss. The training enforces sparsity so that only a small subset of latent units activate per sequence. This encourages each unit to capture a distinct, near-monosemantic genomic feature. (iii) Regression to external readouts: The system fits regression models between the sparse latent units and external biological readouts (e.g., sequence length, motif presence, or experimental measurements) to associate individual neurons with interpretable molecular features.

2.7 Command-line and Web-based User Interface

GENOME-FACTORY offers both a command-line interface (CLI) and a web-based user interface (WebUI) to accommodate a range of user preferences and expertise levels.

Command-line Interface. Users access the command-line interface through a single convenient entry point, `genomefactory-cli`, and define configuration-first workflows with YAML. Users specify tasks, datasets, models, and training or inference settings through compact configuration files. The command-line interface supports four core functionalities: (i) data acquisition and preprocessing with task-specific dataset builders and automated quality control; (ii) model training using full fine-tuning or parameter-efficient methods; (iii) inference for embedding extraction and sequence generation; and (iv) interpretability using the sparse auto-encoder-based Biological Interpreter. The system saves all metrics and artifacts for downstream evaluation and reproducibility.

Web-based User Interface. The WebUI uses Gradio (Abid et al., 2019) to complement the CLI and gives users an intuitive, code-free way to conveniently access all core GENOME-FACTORY features. Users configure data, models, and tasks for training, inference, benchmarking, and interpretability through a clear graphical layout. The interface

shows relevant parameters based on the selected method, applies sensible defaults, and organizes workflows into task-specific tabs. Users launch tasks with a single click and view logs and results in real time within the browser. By hiding the underlying code, the WebUI lets researchers interact with models through a graphical interface.

3 Empirical Study

We comprehensively demonstrate the effectiveness of GENOME-FACTORY through three key dimensions. Each dimension highlights the partial capabilities of six integrated modules: (i) Fine-tuning compatibility across diverse models (Section 3.1): This dimension evaluates the compatibility of three fine-tuning strategies: full, LoRA, and adapter tuning. It covers a range of models. It showcases the functionality of the Model Loader, Model Trainer, and Inference Engine. (ii) Benchmarking downstream performance (Section 3.2): By benchmarking different models on standardized downstream tasks, we analyze their trade-offs between accuracy and computational efficiency. This dimension emphasizes the role of the Benchmark module. (iii) Biological interpretation of learned representations with DNABERT-2 (Section 3.3): We explore how model embeddings capture biological signals, such as correlations with sequence length, to assess interpretability. This dimension demonstrates the capability of the Genome Collector and Biological Interpreter modules.

3.1 Fine-tuning of Diverse Models

We evaluate the six genomic foundation models from the Model Loader using all three fine-tuning strategies from the Model Trainer. Notably, the adapter-based method requires extracting base model embeddings before training. This highlights the functionality of the Inference Engine.

Experimental Setup. We systematically evaluate the training efficiency of full fine-tuning, LoRA, and adapter tuning using the COVID variant prediction task from the GUE benchmark (Zhou et al., 2024). This task involves classifying sequences into one of nine labels. We test six models: HyenaDNA-160k, DNABERT-2, Caduceus-131k, Nucleotide Transformer-500M, EVO-1-131k, and GenomeOcean-100M.

For all experiments, we use AdamW with full-precision (FP32) updates, a fixed learning rate of 3×10^{-5} , batch size 32, and default model settings unless noted. For LoRA tuning, we set the rank to $r = 8$ and the scaling factor to $\alpha = 32$. For most models (DNABERT-2, Caduceus, Nucleotide Transformer, GenomeOcean), we apply LoRA to all feed-forward layers. For HyenaDNA, due to its specialized architecture, we apply LoRA only to the input/output projection layers within the Hyena blocks. For EVO, given

the large size of its feed-forward layers, we target only the key, query, and value matrices in attention blocks. The adapter tuning keeps all base model parameters frozen and adds a trainable multilayer perceptron adapter with a single hidden layer of size 256. We conduct all experiments on a single NVIDIA H100 80GB GPU.

Results. We present a detailed quantitative comparison of training efficiency across three different fine-tuning strategies in Table 2. The table reports the number of trainable parameters, peak GPU memory consumption, and throughput measured in thousands of tokens per second. Among all available methods, adapter tuning consistently shows the highest efficiency. For example, the DNABERT-2 adapter uses only 0.2 million trainable parameters, consumes 2.09 gigabytes of memory, and reaches a throughput of 124,000 tokens per second. In contrast, full fine-tuning for the same model updates 117 million parameters, uses 7.30 gigabytes of memory, and processes only 41,000 tokens per second. The adapter module allows users to adjust its internal structure to meet specific hardware or speed constraints. LoRA also offers strong efficiency gains. For DNABERT-2, it reduces the parameter count to 1.49 million and improves throughput to 44,000 tokens per second while reducing memory usage compared to full fine-tuning. Full fine-tuning remains the most expensive. It requires updating all model parameters, consumes the most GPU memory, and achieves the lowest throughput. Due to its extreme cost, we exclude full fine-tuning for the 7B-parameter EVO model.

To assess scalability, we compare LoRA and full fine-tuning across different model sizes in Table 3. For the Nucleotide Transformer, scaling from 500 million to 2.5 billion parameters lowers the LoRA parameter ratio from 0.9% to 0.5%, increases memory savings from 16% to 32%, and boosts throughput from 1.20 to 1.39 times compared to full tuning. A similar trend appears in GenomeOcean. Scaling from 100 million to 500 million parameters decreases the parameter ratio from 1.5% to 0.7%, increases memory savings from 5% to 21%, and raises throughput from 1.01 to 1.27 times. These results show that LoRA becomes effective as model size grows, making it a strong choice for adapting large genomic models within GENOME-FACTORY. These findings align with expectations and demonstrate the functionality of our Model Loader, Model Trainer, and Inference Engine.

3.2 Benchmarking Different Models

We comprehensively benchmark six genomic foundation models with three different fine-tuning strategies across two benchmark suites. This experiment highlights the role of the Benchmark module by comparing model performance and tuning methods on standardized downstream tasks. This benchmark compares heterogeneous models with different sizes, architectures, pre-training corpora, context lengths,

Table 2. Fine-tuning efficiency with different methods in GENOME-FACTORY. We report the number of trainable parameters, peak GPU memory usage, and throughput (thousands of tokens per second). “—” marks settings we did not evaluate due to computational constraints (e.g., EVO with full fine-tuning). We conduct all experiments on a single NVIDIA H100 (80GB). Due to the large size of EVO (7B) and its computational demands, we exclude full-parameter fine-tuning for this model. “Full” denotes full-parameter fine-tuning, “LoRA” denotes low-rank adaptation, and “Adapter” denotes adapter-based tuning.

Method	HyenaDNA-160k			DNABERT2			Caduceus-131k		
	Trainable params	Mem (GB)	Throughput (KTok/s)	Trainable params	Mem (GB)	Throughput (KTok/s)	Trainable params	Mem (GB)	Throughput (KTok/s)
Full	6.55M	5.75	381.48	117.08M	7.30	40.70	7.73M	6.89	143.55
LoRA	0.10M	4.45	446.05	1.49M	6.18	44.07	0.81M	1.85	394.30
Adapter	0.07M	1.84	1113.32	0.20M	2.09	124.30	0.07M	1.85	467.72

Method	Nucleotide Transformer-500M			EVO-1-131k			GenomeOcean-100M		
	Trainable params	Mem (GB)	Throughput (KTok/s)	Trainable params	Mem (GB)	Throughput (KTok/s)	Trainable params	Mem (GB)	Throughput (KTok/s)
Full	480.45M	18.94	12.08	—	—	—	116.42M	7.39	45.28
LoRA	4.46M	15.95	14.49	0.39M	84.85	6.02	1.70M	7.00	45.88
Adapter	0.33M	4.10	41.28	1.05M	14.74	13.03	0.20M	2.74	137.48

Table 3. Fine-tuning efficiency across different model scales. We report experimental results to clearly illustrate the scalability of LoRA’s efficiency with Nucleotide Transformer (500M/2.5B) and GenomeOcean (100M/500M) as representative examples. We conduct all experiments on a single NVIDIA H100 (80GB). “Full” denotes full-parameter fine-tuning, and “LoRA” denotes low-rank adaptation.

Model	Method	Trainable Params	Peak Mem (GB)	Throughput (KTok/s)
Nucleotide Transformer-500M	Full	480.45 M	18.94	12.08
	LoRA	4.46 M	15.95	14.49
Nucleotide Transformer-2.5B	Full	2.54 B	63.95	2.40
	LoRA	11.86 M	43.33	3.34
GenomeOcean-100M	Full	116.42 M	7.39	45.28
	LoRA	1.70 M	7.00	45.88
GenomeOcean-500M	Full	534.83 M	19.58	11.85
	LoRA	3.97 M	15.48	15.07

and tokenizers. Thus, the results reflect practical model performance within GENOME-FACTORY, not intrinsic architectural superiority. Controlled architectural conclusions would require matched scale, data, and training budgets.

Experimental Setup. We evaluate model performance on tasks from two widely used sources: the GUE benchmark (Zhou et al., 2024) and Genomic Benchmarks (Grešová et al., 2023). For most benchmark tasks, we report reliable test set performance with the Matthews correlation coefficient. For the COVID variant prediction task, we follow the benchmark protocol and report the F1 score. We evaluate the same six models in Section 3.1: HyenaDNA-160k, DNABERT-2, Caduceus-131k, Nucleotide Transformer-500M, EVO-1-131k, and GenomeOcean-100M. To provide a robust reference for the computational overhead of foundation models, we compare against several non-foundation-model baselines, including Support Vector Machines and

Random Forests trained on 4-mer frequency features, a Convolutional Neural Network operating on one-hot encoded sequences, Position Weight Matrix (PWM)-based motif-scanning pipelines paired with Random Forest or linear SVM classifiers, and Hidden Markov Model (HMM)-based motif-finding approaches. For each experiment, we consistently use random seeds 14, 28, and 42, and report the mean score along with the standard deviation.

We fine-tune all models with a learning rate of 3.0×10^{-5} and the AdamW optimizer in full-precision (FP32) mode. For Caduceus, we use a higher learning rate of 1.0×10^{-3} to ensure training stability. For LoRA tuning, we use rank $r = 8$ and scaling factor $\alpha = 32$. Similar to Section 3.1, we apply LoRA to all feed-forward layers in DNABERT-2, Caduceus, Nucleotide Transformer, and GenomeOcean. For HyenaDNA, we apply LoRA only to the input and output projection layers within the Hyena blocks. For the EVO

model, we target the key, query, and value matrices in each attention block, due to the large size of feed-forward layer parameters. The adapter tuning freezes all base models and appends a trainable multilayer perceptron adapter with a single hidden layer of size 256. We perform training on a single NVIDIA H100 80GB GPU for most models with a batch size of 32. For Nucleotide Transformer-500M, due to its larger memory footprint during full-parameter tuning, we fine-tune it with distributed data parallel across two NVIDIA H100 80GB GPUs. This also demonstrates the functionality of our distributed training.

Results. We show results on downstream tasks in Table 4. Due to the large size of the EVO model (7B parameters) and its high computational cost, we omit full fine-tuning results for this model. We report the averaged Matthews correlation coefficient across all evaluated datasets. Except for the COVID subset of the GUE benchmark, we follow the original protocol and use the macro-averaged F1 score. The overall GUE score is computed by averaging this F1 value with the correlation coefficients from the remaining tasks. We observe that GenomeOcean-100M achieves the strongest performance under full fine-tuning on the GUE, while DNABERT-2 performs best on the Genomic Benchmarks. Parameter-efficient methods remain competitive. For example, LoRA sometimes matches full fine-tuning, as seen with GenomeOcean-100M on Genomic Benchmarks (68.26 vs. 66.28). While Adapter method underperforms LoRA in most cases, it narrows the gap and even surpasses LoRA for Caduceus. This shows that a task-aware adapter improves the task performance.

We visualize the trade-offs between predictive performance and computational efficiency for DNABERT-2, HyenaDNA, and Nucleotide Transformer in Figure 2 under three fine-tuning strategies: full fine-tuning, low-rank adaptation, and adapter-based methods. The figure reports memory usage, training throughput, and GUE scores. Adapter tuning reduces memory usage and boosts throughput across all models, though it sacrifices some predictive performance. In contrast, full fine-tuning and low-rank adaptation yield higher GUE scores but require more compute. This comparison highlights the flexible trade-off space between efficiency and accuracy with GENOME-FACTORY. Overall, these results demonstrate both the functionality and practical utility of the Benchmark module.

Analysis of Performance Differences. The observed performance gaps likely reflect a combination of model scale, architecture, and pre-training data rather than a single factor. Larger models can provide greater representational capacity, but size alone does not explain the results: for example, EVO-1-131k is much larger than the other models but does not consistently achieve the best downstream performance under parameter-efficient tuning. Architec-

ture and tokenization also matter because HyenaDNA, Caduceus, DNABERT-2, Nucleotide Transformer, EVO, and GenomeOcean encode sequence context through different mechanisms and token granularities. Pre-training data further affects transfer performance, since models trained on different species, genomic regions, or metagenomic corpora may capture different biological signals. Therefore, we interpret Table 4 as an empirical comparison of practical model choices within GENOME-FACTORY, rather than a controlled ablation of model architecture.

3.3 Biological Interpretation of Genomic Models

We demonstrate the capabilities of the Genome Collector and Biological Interpreter modules by analyzing model embeddings through sparse autoencoder-based interpretability.

Experimental Setup. We use the Genome Collector to automate genomic data preparation. The Data Acquisition component downloads genome sequences for two organisms from NCBI: *Arabidopsis thaliana* and *Bos taurus*. We save all downloaded files in a unified directory. After acquisition, the Data Preprocessing component segments each genome into 1,000 sequences of random length between 500 and 1,000 base pairs, yielding 2,000 total segments. We then apply standard quality control procedures, including GC-content correction and removal of N nucleotides. Finally, we construct a labeled dataset by assigning each sequence a label corresponding to its sequence length.

To explore representation learning, we train a sparse autoencoder (SAE) with a hidden dimension of 4,096 using model representations and reconstruction losses. After training, we use the SAE latent activations to perform regression on the sequence-length labels, allowing us to identify latent features associated with sequence length.

To further validate whether SAE neurons capture biologically meaningful regulatory features, we construct a CTCF-binding dataset from ENCODE K562 ChIP-seq peaks. The dataset contains 20,000 IDR peaks as positive examples and length-matched background sequences as negative examples, resulting in 39,024 total sequences. We train a logistic regression probe on SAE latent activations to classify CTCF-bound sequences versus background sequences.

Results. In the sequence-length analysis, we find that `latent_382`, `latent_519`, and `latent_3519` exhibit strong correlations with sequence length, suggesting that individual SAE latent dimensions encode interpretable sequence-level properties. In the CTCF-binding analysis, the logistic regression probe achieves an accuracy of 0.7881 for distinguishing CTCF-bound sequences from background sequences. We further assess neuron-level importance by individually zeroing the most influential SAE latent dimensions and measuring the resulting drop in classifica-

Table 4. Benchmark across models and tuning methods. We report results on both the GUE benchmark and Genomic Benchmarks. We include traditional baselines to establish a performance floor for foundation-model evaluation: a Convolutional Neural Network (CNN), Random Forest, and Support Vector Machine (SVM) classifiers trained on 4-mer frequency features, Position Weight Matrix (PWM)-based motif-scanning pipelines paired with Random Forest or linear SVM classifiers, and Hidden Markov Model (HMM)-based motif-finding methods. “—” marks settings we did not evaluate due to computational constraints. We exclude EVO (7B) from full fine-tuning due to the high computation cost. Three baselines are trained directly on downstream tasks and thus do not utilize LoRA or Adapter modules. “Full” denotes full-parameter, “LoRA” denotes low-rank adaptation, and “Adapter” denotes adapter-based tuning.

Model	Size	GUE			Genomic Benchmarks		
		Full	LoRA	Adapter	Full	LoRA	Adapter
HyenaDNA-160k	7M	59.91 $\pm$ 0.22	50.95 $\pm$ 0.23	25.00 $\pm$ 0.37	66.71 $\pm$ 0.18	61.26 $\pm$ 0.29	36.90 $\pm$ 0.42
Caduceus-131k	8M	50.07 $\pm$ 1.61	34.70 $\pm$ 0.19	38.61 $\pm$ 0.33	65.38 $\pm$ 0.22	42.10 $\pm$ 0.36	47.74 $\pm$ 0.27
DNABERT-2	117M	65.25 $\pm$ 0.25	48.39 $\pm$ 0.11	40.19 $\pm$ 0.17	71.97 $\pm$ 0.20	64.58 $\pm$ 0.25	48.96 $\pm$ 0.31
GenomeOcean-100M	117M	65.52 $\pm$ 0.57	59.35 $\pm$ 0.23	46.65 $\pm$ 0.04	68.26 $\pm$ 0.26	66.28 $\pm$ 0.19	53.60 $\pm$ 0.21
Nucleotide Transformer-500M	480M	57.63 $\pm$ 0.26	52.96 $\pm$ 0.13	32.01 $\pm$ 0.48	67.99 $\pm$ 0.24	64.68 $\pm$ 0.30	43.95 $\pm$ 0.28
EVO-1-131k	7B	—	44.97 $\pm$ 0.79	30.08 $\pm$ 0.22	—	51.33 $\pm$ 0.41	33.69 $\pm$ 0.34
CNN	—	45.07 $\pm$ 1.42	—	—	56.58 $\pm$ 1.50	—	—
Random Forest	—	51.79 $\pm$ 0.12	—	—	61.25 $\pm$ 0.50	—	—
Support Vector Machine	—	57.01 $\pm$ 0.00	—	—	59.89 $\pm$ 0.00	—	—
PWM + Random Forest	—	56.85 $\pm$ 0.47	—	—	62.07 $\pm$ 0.52	—	—
PWM + Linear SVM	—	55.18 $\pm$ 0.22	—	—	60.14 $\pm$ 0.38	—	—
Hidden Markov Model	—	50.21 $\pm$ 0.35	—	—	56.92 $\pm$ 0.61	—	—

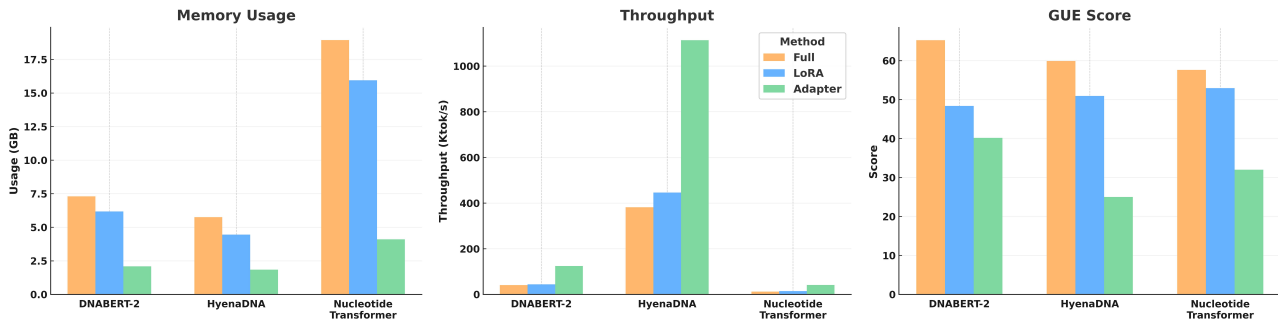


Figure 2. Trade-off between tuning efficiency and performance. The figure shows memory usage in gigabytes (GB), throughput in kilotokens per second (Ktok/s), and averaged scores on the GUE benchmark for three models: DNABERT-2, HyenaDNA-160k, and Nucleotide Transformer-500M. We report results for full-tuning (Full), low-rank adaptation (LoRA), and adapter-based fine-tuning (Adapter). The results highlight the trade-offs between resource efficiency and predictive performance.

tion accuracy. Zeroing `latent_2939`, `latent_3184`, and `latent_10` reduces accuracy by 0.0225, 0.0210, and 0.0196, respectively. These results show that specific SAE latent dimensions capture biologically meaningful, motif-centric regulatory features.

Together, these analyses demonstrate that the Biological Interpreter can move beyond black-box prediction by linking individual latent features to measurable genomic properties and regulatory signals. This provides stronger support for the mechanistic interpretation of genomic foundation models and establishes an open-source framework for connecting model representations with interpretable insights.

4 Conclusion and Future Work

We introduce GENOME-FACTORY, the first unified and modular Python framework for streamlining the tuning, deploy-

ment, and interpretation of genomic models. GENOME-FACTORY offers an end-to-end pipeline that integrates six key components: Genome Collector for acquiring and pre-processing data; Model Loader for accessing genomic models; Model Trainer for fine-tuning models tailored to specific downstream tasks; Inference Engine for embedding extraction and sequence generation; Benchmarking for evaluating model performance; and Biological Interpreter for interpretability via sparse autoencoders. Our experiments demonstrate the utility of GENOME-FACTORY on key genomic downstream tasks using multiple training methods across diverse models. These results highlight its end-to-end usability for real-world analysis. GENOME-FACTORY lowers the technical barrier to using large-scale models in genomics research. It makes these powerful tools more accessible to the broader research community. We discuss the limitations in Appendix B, related works in Appendix C, and provide usage examples of GENOME-FACTORY in Appendix G.

Acknowledgements

Weimin Wu would like to thank the anonymous reviewers and program chairs for constructive comments. JH is partially supported by the Walter P. Murphy Fellowship and the Terminal Year Fellowship (Paul K. Richter Memorial Award) of Northwestern University. Han Liu is partially supported by NIH R01LM1372201, NSF AST-2421845, Simons Foundation MPS-AI-00010513, AbbVie, Dolby, and Chan Zuckerberg Biohub Chicago Spoke Award. Zhong Wang’s work conducted at the U.S. Department of Energy Joint Genome Institute (<https://ror.org/04xm1d337>), a DOE Office of Science User Facility, is supported by the Office of Science of the U.S. Department of Energy operated under Contract No. DE-AC02-05CH11231. This research was supported in part through the computational resources and staff contributions provided for the Quest high performance computing facility at Northwestern University which is jointly supported by the Office of the Provost, the Office for Research, and Northwestern University Information Technology. The content is solely the responsibility of the authors and does not necessarily represent the official views of the funding agencies.

Impact Statement

GENOME-FACTORY provides open-source infrastructure that bridges complex machine learning architectures and practical biological discovery by unifying the genomic model lifecycle, from automated data preprocessing to sparse auto-encoder-based interpretation. By offering zero-code command-line and web interfaces, GENOME-FACTORY lowers the technical barrier to advanced genomic modeling and enables researchers with varying computational expertise to fine-tune, benchmark, and interpret genomic foundation models. This can accelerate research in personalized medicine, evolutionary biology, and therapeutic design while supporting open, reproducible, and responsible scientific innovation.

At the same time, democratized access to generative sequence models introduces potential dual-use biosecurity risks, including the in silico generation of pathogenic, virulence-associated, or heavily modified viral sequences. Responsible deployment therefore requires safeguards beyond user-level ethical guidelines. Future versions of GENOME-FACTORY can incorporate software-level guardrails, including sequence screening API integration, pathogen- or virulence-associated motif blocklists, risk-aware generation filters, request logging and auditing, and configurable access controls for high-risk generation tasks. These safeguards can help flag or restrict potentially unsafe outputs while preserving legitimate research use.

References

- Abubakar Abid, Ali Abdalla, Ali Abid, Dawood Khan, Abdulrahman Alfozan, and James Zou. Gradio: Hassle-free sharing and testing of ml models in the wild. *arXiv preprint arXiv:1906.02569*, 2019.
- Josh Achiam, Steven Adler, Sandhini Agarwal, Lama Ahmad, Ilge Akkaya, Florencia Leoni Aleman, Diogo Almeida, Janko Altenschmidt, Sam Altman, Shyamal Anadkat, et al. Gpt-4 technical report. *arXiv preprint arXiv:2303.08774*, 2023.
- Giovanna Ambrosini, Romain Groux, and Philipp Bucher. Pwmscan: a fast tool for scanning entire genomes with a position-specific weight matrix. *Bioinformatics*, 34(14): 2483–2484, 2018.
- Yuvanesh Anand, Zach Nussbaum, Brandon Duderstadt, Benjamin Schmidt, and Andriy Mulyar. Gpt4all: Training an assistant-style chatbot with large scale data distillation from gpt-3.5-turbo. *GitHub* <https://github.com/nomic-ai/gpt4all>, 2023.
- Žiga Avsec, Natasha Latysheva, Jun Cheng, Guido Novati, Kyle R Taylor, Tom Ward, Clare Bycroft, Lauren Nicolaisen, Eirini Arvaniti, Joshua Pan, et al. Advancing regulatory variant effect prediction with alphagenome. *Nature*, 649(8099):1206–1218, 2026.
- Garyk Brixi, Matthew G Durrant, Jerome Ku, Mohsen Naghipourfar, Michael Poli, Gwanggyu Sun, Greg Brockman, Daniel Chang, Alison Fanton, Gabriel A Gonzalez, et al. Genome modelling and design across all domains of life with evo 2. *Nature*, 652(8112):1349–1361, 2026.
- Wenduo Cheng, Zhenqiao Song, Yang Zhang, Shike Wang, Danqing Wang, Muyu Yang, Lei Li, and Jian Ma. Dna-longbench: a benchmark suite for long-range dna prediction tasks. *Nature Communications*, 16(1):10108, 2025.
- Gheorghe Comanici, Eric Bieber, Mike Schaeckermann, Ice Pasupat, Noveen Sachdeva, Inderjit Dhillon, Marcel Blisstein, Ori Ram, Dan Zhang, Evan Rosen, et al. Gemini 2.5: Pushing the frontier with advanced reasoning, multimodality, long context, and next generation agentic capabilities. *arXiv preprint arXiv:2507.06261*, 2025.
- Micaela Elisa Consens, Ben Li, Anna R Poetsch, and Stephen Gilbert. Genomic language models could transform medicine but not yet. *NPJ Digit. Med.*, 8(1):212, April 2025.
- Hugo Dalla-Torre, Liam Gonzalez, Javier Mendoza-Revilla, Nicolas Lopez Carranza, Adam Henryk Grzywaczewski, Francesco Oteri, Christian Dallago, Evan Trop, Bernardo P de Almeida, Hassan Sirelkhatim, et al. Nucleotide transformer: building and evaluating

- robust foundation models for human genomics. *Nature Methods*, 22(2):287–297, 2025.
- Tri Dao, Dan Fu, Stefano Ermon, Atri Rudra, and Christopher Ré. Flashattention: Fast and memory-efficient exact attention with io-awareness. *Advances in Neural Information Processing Systems*, 35:16344–16359, 2022.
- Pieter-Tjerk De Boer, Dirk P Kroese, Shie Mannor, and Reuven Y Rubinstein. A tutorial on the cross-entropy method. *Annals of operations research*, 134(1):19–67, 2005.
- Shizhe Diao, Rui Pan, Hanze Dong, Kashun Shum, Jipeng Zhang, Wei Xiong, and Tong Zhang. Lmflow: An extensible toolkit for finetuning and inference of large foundation models. In *NAACL (Demonstrations)*, 2024.
- Vage Egiazarian, Andrei Panferov, Denis Kuznedelev, Elias Frantar, Artem Babenko, and Dan Alistarh. Extreme compression of large language models via additive quantization. In *Proceedings of the 41st International Conference on Machine Learning*, pages 12284–12303, 2024.
- Zijing Gao, Qiao Liu, Wanwen Zeng, Rui Jiang, and Wing Hung Wong. EpiGePT: a pretrained transformer-based language model for context-specific human epigenomics. *Genome Biol.*, 25(1):310, December 2024.
- Lewis Y Geer, Aron Marchler-Bauer, Renata C Geer, Lianyi Han, Jane He, Siqian He, Chunlei Liu, Wenyao Shi, and Stephen H Bryant. The ncbi biosystems database. *Nucleic acids research*, 38(suppl_1):D492–D496, 2010.
- Katarína Grešová, Vlastimil Martinek, David Čechák, Petr Šimeček, and Panagiotis Alexiou. Genomic benchmarks: a collection of datasets for genomic sequence classification. *BMC Genomic Data*, 24(1):25, 2023.
- Albert Gu and Tri Dao. Mamba: Linear-time sequence modeling with selective state spaces. *First Conference on Language Modeling*, 2024.
- Daya Guo, Dejian Yang, Haowei Zhang, Junxiao Song, Peiyi Wang, Qihao Zhu, Runxin Xu, Ruoyu Zhang, Shitong Ma, Xiao Bi, et al. Deepseek-r1: Incentivizing reasoning capability in llms via reinforcement learning. *arXiv preprint arXiv:2501.12948*, 2025.
- Ruidan He, Linlin Liu, Hai Ye, Qingyu Tan, Bosheng Ding, Liying Cheng, Jiawei Low, Lidong Bing, and Luo Si. On the effectiveness of adapter-based tuning for pretrained language model adaptation. In *Proceedings of the 59th Annual Meeting of the Association for Computational Linguistics and the 11th International Joint Conference on Natural Language Processing (Volume 1: Long Papers)*, pages 2208–2222, 2021.
- Pin-Lun Hsu, Yun Dai, Vignesh Kothapalli, Qingquan Song, Shao Tang, Siyu Zhu, Steven Shimizu, Shivam Sahni, Haowen Ning, and Yanning Chen. Liger kernel: Efficient triton kernels for llm training. *arXiv preprint arXiv:2410.10989*, 2024.
- Edward J Hu, Yelong Shen, Phillip Wallis, Zeyuan Allen-Zhu, Yanzhi Li, Shean Wang, Lu Wang, Weizhu Chen, et al. Lora: Low-rank adaptation of large language models. *International Conference on Learning Representations*, 2022.
- Yunha Hwang, Andre L Cornman, Elizabeth H Kellogg, Sergey Ovchinnikov, and Peter R Girguis. Genomic language model predicts protein co-regulation and function. *Nat. Commun.*, 15(1):2880, April 2024.
- Nilah Ioannidis et al. Guanine v1. 0: Benchmark datasets for genomic ai sequence-to-function models. In *Machine Learning in Computational Biology*, pages 250–266. PMLR, 2024.
- Shen Li, Yanli Zhao, Rohan Varma, Omkar Salpekar, Pieter Noordhuis, Teng Li, Adam Paszke, Jeff Smith, Brian Vaughan, Pritam Damania, et al. Pytorch distributed: Experiences on accelerating data parallel training. *Proceedings of the VLDB Endowment*, 13(12), 2020.
- Shenggui Li, Hongxin Liu, Zhengda Bian, Jiarui Fang, Haichen Huang, Yuliang Liu, Boxiang Wang, and Yang You. Colossal-ai: A unified deep learning system for large-scale parallel training. In *Proceedings of the 52nd International Conference on Parallel Processing*, pages 766–775, 2023.
- Eric Nguyen, Michael Poli, Marjan Faizi, Armin Thomas, Michael Wornow, Callum Birch-Sykes, Stefano Marsaroli, Aman Patel, Clayton Rabideau, Yoshua Bengio, et al. Hyenadna: Long-range genomic sequence modeling at single nucleotide resolution. *Advances in Neural Information Processing Systems*, 36:43177–43201, 2023.
- Eric Nguyen, Michael Poli, Matthew G Durrant, Brian Kang, Dhruva Katrekar, David B Li, Liam J Bartie, Armin W Thomas, Samuel H King, Garyk Brix, et al. Sequence modeling and design from molecular to genome scale with evo. *Science*, 386(6723):ead09336, 2024.
- Keiron O’shea and Ryan Nash. An introduction to convolutional neural networks. *arXiv preprint arXiv:1511.08458*, 2015.
- Marius-Constantin Popescu, Valentina E Balas, Liliana Perescu-Popescu, and Nikos Mastorakis. Multilayer perceptron and neural networks. *WSEAS Transactions on Circuits and Systems*, 8(7):579–588, 2009.

- Zhaohui S Qin, Jianjun Yu, Jincheng Shen, Christopher A Maher, Ming Hu, Shanker Kalyana-Sundaram, Jindan Yu, and Arul M Chinnaiyan. Hpeak: an hmm-based algorithm for defining read-enriched regions in chip-seq data. *BMC bioinformatics*, 11:1–13, 2010.
- Mark Saroufim, Yotam Perlitz, Leshem Choshen, Luca Antiga, Greg Bowyer, Christian Puhersch, Driss Guesous, Supriya Rao, Geeta Chauhan, Ashvini Kumar, et al. Neurips 2023 llm efficiency fine-tuning competition. *arXiv preprint arXiv:2503.13507*, 2025.
- Yair Schiff, Chia Hsiang Kao, Aaron Gokaslan, Tri Dao, Albert Gu, and Volodymyr Kuleshov. Caduceus: Bi-directional equivariant long-range dna sequence modeling. In *International Conference on Machine Learning*, pages 43632–43648. PMLR, 2024.
- Mark D Schluchter. Mean square error. *Encyclopedia of Biostatistics*, 5, 2005.
- Maojiang Su, Po-Chung Hsieh, Weimin Wu, Mingcheng Lu, Jiunhau Chen, Jerry Yao-Chieh Hu, and Han Liu. Discrete flow matching policy optimization. *arXiv preprint arXiv:2604.06491*, 2026.
- Hugo Touvron, Thibaut Lavril, Gautier Izacard, Xavier Martinet, Marie-Anne Lachaux, Timothée Lacroix, Baptiste Rozière, Naman Goyal, Eric Hambro, Faisal Azhar, Aurelien Rodriguez, Armand Joulin, Edouard Grave, and Guillaume Lample. Llama: Open and efficient foundation language models, 2023.
- Evan Trop, Yair Schiff, Edgar Mariano Marroquin, Chia Hsiang Kao, Aaron Gokaslan, McKinley Polen, Mingyi Shao, Bernardo P de Almeida, Thomas Pierrot, Yang I Li, et al. The genomics long-range benchmark: advancing dna language models. 2024.
- Yizhong Wang, Hamish Ivison, Pradeep Dasigi, Jack Hessel, Tushar Khot, Khyathi Chandu, David Wadden, Kelsey MacMillan, Noah A Smith, Iz Beltagy, et al. How far can camels go? exploring the state of instruction tuning on open resources. *Advances in Neural Information Processing Systems*, 36:74764–74786, 2023.
- Thomas Wolf, Lysandre Debut, Victor Sanh, Julien Chaumond, Clement Delangue, Anthony Moi, Pierric Cistac, Tim Rault, Rémi Louf, Morgan Funtowicz, et al. Huggingface’s transformers: State-of-the-art natural language processing. *arXiv preprint arXiv:1910.03771*, 2019.
- Weimin Wu, Alexander C Furnas, Eddie Yang, Gefei Liu, Akhil Pandey Akella, Xuefeng Song, Dashun Wang, and Han Liu. Sci2pol: Evaluating and fine-tuning llms on scientific-to-policy brief generation. *arXiv preprint arXiv:2509.21493*, 2025.
- Renrui Zhang, Jiaming Han, Aojun Zhou, Xiangfei Hu, Shilin Yan, Pan Lu, Hongsheng Li, Peng Gao, and Yu Qiao. Llama-adapter: Efficient fine-tuning of language models with zero-init attention. *International Conference on Learning Representations*, 2024.
- Lianmin Zheng, Wei-Lin Chiang, Ying Sheng, Siyuan Zhuang, Zhanghao Wu, Yonghao Zhuang, Zi Lin, Zhuohan Li, Dacheng Li, Eric Xing, et al. Judging llm-as-a-judge with mt-bench and chatbot arena. *Advances in Neural Information Processing Systems*, 36:46595–46623, 2023.
- Yaowei Zheng, Richong Zhang, Junhao Zhang, YeYanhan YeYanhan, and Zheyang Luo. Llamafactory: Unified efficient fine-tuning of 100+ language models. In *Proceedings of the 62nd Annual Meeting of the Association for Computational Linguistics (Volume 3: System Demonstrations)*, pages 400–410, 2024.
- Zhihan Zhou, Yanrong Ji, Weijian Li, Pratik Dutta, Ramana Davuluri, and Han Liu. Dnabert-2: Efficient foundation model and benchmark for multi-species genome. In *International Conference on Learning Representations*, 2024.
- Zhihan Zhou, Robert Riley, Satria Kautsar, Weimin Wu, Rob Egan, Steven Hofmeyr, Shira Goldhaber-Gordon, Mutian Yu, Harrison Ho, Fengchen Liu, et al. Genomeocean: An efficient genome foundation model trained on large-scale metagenomic assemblies. *bioRxiv*, pages 2025–01, 2025a.
- Zhihan Zhou, Weimin Wu, Harrison Ho, Jiayi Wang, Lizhen Shi, Ramana V Davuluri, Zhong Wang, and Han Liu. Dnabert-s: Pioneering species differentiation with species-aware dna embeddings. *Bioinformatics*, 41:i255–i264, 2025b.

Appendix

A	LLM Usage Acknowledgement	14
B	Limitations	15
C	Related work	16
C.1	Genomic Foundation Models	16
C.2	Libraries for Language Models	16
D	Experimental Details	17
E	Details of Supported Models	18
F	Additional Benchmarking on Broader Genomic Tasks	19
G	Usage Examples of GENOME-FACTORY	20
G.1	Data Downloading	20
G.2	Data Processing	20
G.3	Model Training	21
G.4	Model Inference	22
G.5	Biological Interpretation	22
G.6	Usage via WebUI	23

A LLM Usage Acknowledgement

Large language models (LLMs) are used in this project as general-purpose assistive tools to support writing and coding tasks. We detail their specific roles below.

- **Manuscript Writing:** We use **GPT-4o** to polish the manuscript, including grammar correction, phrasing refinement, and improving clarity throughout the text.
- **Code Development:** We use **GPT-4o** for debugging, code generation, and resolving implementation issues during the library development.

All LLM-assisted content is reviewed and curated by the authors. We take full responsibility for the final codebase and manuscript.

B Limitations

Although GENOME-FACTORY provides a unified and practical framework for genomic foundation model training and evaluation, it does not address every advanced research scenario. We identify the following limitations.

- **Scalability of full fine-tuning.** Full fine-tuning of very large genomic foundation models remains computationally expensive. For models above several hundred million parameters, such as EVO-7B, full fine-tuning is difficult to perform on a single 80GB GPU. Therefore, GENOME-FACTORY currently emphasizes parameter-efficient adaptation methods, such as LoRA and adapter tuning, for large-scale models. Future versions will incorporate more memory- and compute-efficient training strategies, including quantization (Egiazarian et al., 2024), optimized kernels such as the Liger kernel (Hsu et al., 2024), and distributed training support.
- **Limitation of context truncation.** GENOME-FACTORY currently truncates sequences that exceed a model’s maximum context length while leaving shorter sequences unchanged. This can remove biologically meaningful elements and may bias cross-model comparisons, since short-context models observe less of the original sequence. In our current GUE and Genomic Benchmark experiments, this issue has limited impact because only one task in each benchmark contains sequences longer than 512 bp. However, it becomes important for long-range benchmarks such as DNALONGBENCH (Cheng et al., 2025), where sequences can reach 114k bp. Future versions will support sliding-window evaluation and multi-window aggregation to better handle long genomic regions.
- **Limited integration of classical computational biology methods.** We conduct experiments using a unified genomic foundation model pipeline and do not yet systematically incorporate traditional computational biology techniques, such as PWM scanning (Ambrosini et al., 2018) and HMM-based motif finding (Qin et al., 2010). These methods remain strong and interpretable baselines for many sequence analysis tasks. Future work will integrate classical methods into the same evaluation framework to enable fair comparison and hybrid modeling.
- **Validation of multi-task learning.** GENOME-FACTORY now supports multi-task learning, but this functionality has not yet been extensively validated across diverse task combinations, label structures, and genomic modalities. In the current version, single-task training remains the default and most thoroughly evaluated setting. Future work will systematically study when multi-task learning improves genomic representation learning and when task interference may reduce performance.
- **Coverage of fine-tuning strategies.** We currently include several commonly used fine-tuning methods, but the design space of efficient adaptation remains much broader. Future versions will incorporate additional training strategies and computational acceleration techniques to better support large-scale genomic model development.

C Related work

In the following, we first discuss the genomic foundation models in Appendix C.1. Next, we discuss the existing libraries for natural language models in Appendix C.2.

C.1 Genomic Foundation Models

Several powerful genomic foundation models have recently emerged to decode the complex language of DNA (Avsec et al., 2026; Brixi et al., 2026; Su et al., 2026; Zhou et al., 2025b; Nguyen et al., 2024; Zhou et al., 2024). For the discriminative model domain, HyenaDNA (Nguyen et al., 2023) employs implicit convolution layers. This enables million-token contexts at single-nucleotide resolution to capture complex regulatory interactions and support in-context species classification. DNABERT-2 (Zhou et al., 2024) employs byte pair encoding and a refined transformer architecture. It improves tasks such as epigenomic mark prediction and transcription factor binding. Caduceus (Schiff et al., 2024) leverages the Mamba architecture (Gu and Dao, 2024) with reverse-complement equivariance to improve long-range variant-effect prediction. Nucleotide Transformer (Dalla-Torre et al., 2025) scales to 2.5B parameters with 6-mer tokenization. It achieves strong performance in chromatin-feature prediction and functional variant prioritization. For the generative model domain, Evo (Nguyen et al., 2024) introduces a 7B-parameter generative model to extend beyond embedding extraction. This enables genome-scale predictions across biomolecular modalities. GenomeOcean (Zhou et al., 2025a) further improves generative efficiency for metagenomic sequence synthesis. It advances applications in synthetic biology. However, diverse models use a wide range of environments with heterogeneous dependencies and configurations. This lack of standardization makes it difficult to load, fine-tune, or compare models. Such fragmentation increases the technical burden on users and limits the accessibility of genomic models. To address this, we introduce GENOME-FACTORY, the first Python framework to unify and streamline the end-to-end genomic model workflow.

C.2 Libraries for Language Models

In parallel, the language community (Comanici et al., 2025; Guo et al., 2025; Wu et al., 2025; Achiam et al., 2023; Touvron et al., 2023) has developed numerous frameworks to streamline the adaptation and fine-tuning of language models. These toolkits target different stages of the language model lifecycle. For example, LLaMA-Adapter (Zhang et al., 2024) improves fine-tuning efficiency, while GPT4All (Anand et al., 2023) enables practical model training and efficient inference on widely available consumer-grade hardware. Other frameworks address specific training challenges or model architectures: Colossal-AI (Li et al., 2023) introduces advanced parallelism strategies for efficient large-scale distributed training, FastChat (Zheng et al., 2023) provides specialized tools for building dialogue agents, and Open-Instruct (Wang et al., 2023) standardizes methodologies for instruction tuning. Flexibility, and domain specialization have also emerged as key priorities. LitGPT (Saroufim et al., 2025) adopts a modular design to support diverse generative model training paradigms, while LMFlow (Diao et al., 2024) helps researchers train language models for specific domains. Finally, LLaMA-Factory (Zheng et al., 2024) further unifies this ecosystem by integrating multiple efficient fine-tuning techniques into a single, comprehensive toolkit. However, these frameworks do not translate to the genomic domain. Genomic models require specialized data formats, biology-informed objectives, domain-specific benchmarks, and meaningful biological insights. GENOME-FACTORY fills this gap with tools tailored for genomic models. It supports data collection, model tuning, inference, benchmarking, and biological interpretation. Notably, the Biological Interpreter is an open-source tool to decode the hidden internal representations of genomic models.

D Experimental Details

In this section, we provide the additional experimental details for Section 3.2.

Traditional Baseline Models We include six baselines: Support Vector Machine (SVM), Random Forest (RF), convolutional neural network (CNN), PWM + RF, PWM + linear SVM, and Hidden Markov Model (HMM). For all baselines, hyperparameters are selected using the validation set only, and the test set is evaluated once using the best validation configuration. All experiments use the same train/validation/test splits as the genomic foundation model experiments.

For SVM and RF, we extract normalized 6-mer frequency features with a stride of 1. Reverse complements are treated as distinct features, and each count vector is normalized by the total number of extracted k-mers. For CNN, we use one-hot encoded DNA sequences as input. For PWM-based baselines, we scan each sequence with position weight matrices and use the resulting motif-score features as input to either RF or linear SVM classifiers. For the HMM baseline, we train an HMM-based motif model and use the resulting sequence likelihood or motif-enrichment scores for downstream classification. Traditional baselines are implemented using *scikit-learn* and *PyTorch*. See the detailed hyperparameter settings in Table 5.

Table 5. **Hyperparameter settings for traditional baseline models.** Hyperparameters are selected on the validation set, and the test set is evaluated once using the best validation configuration.

Model	Input features	Search space	Selected configuration
SVM	6-mer frequency; stride = 1; reverse complements treated separately; normalized by total k-mer count	Kernel $\in \{\text{linear}, \text{rbf}\}$; $C \in \{0.1, 1, 10, 100\}$; for RBF, $\gamma \in \{\text{scale}, 10^{-3}, 10^{-2}, 10^{-1}\}$	Kernel = rbf; $C = 10$; $\gamma = \text{scale}$
RF	Same 6-mer frequency features as SVM	Trees $\in \{100, 300, 500\}$; max depth $\in \{\text{None}, 10, 20, 40\}$; min samples split $\in \{2, 5, 10\}$; min samples leaf $\in \{1, 2, 4\}$; max features $\in \{\text{sqrt}, \log_2, 0.5\}$	Trees = 500; max depth = 20; min samples split = 2; min samples leaf = 1; max features = sqrt
CNN	One-hot encoded DNA sequence	3 convolutional layers; channels (128, 256, 256); kernel sizes (7, 5, 3); ReLU; global max pooling; FC hidden dim = 256; dropout $\in \{0.3, 0.5\}$; learning rate $\in \{10^{-4}, 5 \times 10^{-4}, 10^{-3}\}$; batch size $\in \{32, 64, 128\}$; weight decay $\in \{0, 10^{-5}, 10^{-4}\}$	Adam; learning rate = 10^{-3} ; batch size = 64; dropout = 0.5; weight decay = 10^{-5} ; max epochs = 30; early stopping patience = 5
PWM + RF	PWM scores motif-scanning	Same RF search space as above	Trees = 500; max depth = 20; min samples split = 2; min samples leaf = 1; max features = sqrt
PWM + SVM	PWM scores motif-scanning	Linear SVM with $C \in \{0.1, 1, 10, 100\}$	Linear kernel; $C = 10$
HMM	HMM-derived sequence likelihood	Number of hidden states $\in \{2, 4, 8\}$; covariance/-transition smoothing $\in \{10^{-4}, 10^{-3}, 10^{-2}\}$	Hidden states = 4; smoothing = 10^{-3}

E Details of Supported Models

The Table 6 show GENOME-FACTORY ’s list of supported models.

Table 6. Supported models in GENOME-FACTORY with their available variants. Models vary by either parameter size (e.g., GenomeOcean: 100M/500M/4B) or input sequence length (e.g., Hyenadna: 1K to 1M). "Variant Type" specifies the axis of variation, and "Variants" lists the available options.

Model	Variant Type	Variants
Hyenadna	Sequence Length	1K/16K/32K/160K/450K/1M
DNABERT-2	Parameter Size	117M
Caduceus	Sequence Length	1K/131K
Nucleotide Transformer	Parameter Size	50M/100M/250M/500M/1B/2.5B
EVO	Sequence Length	8K/131K
GenomeOcean	Parameter Size	100M/500M/4B

F Additional Benchmarking on Broader Genomic Tasks

To further assess the generalizability of GENOME-FACTORY beyond the benchmark suites reported in Section 3.2, we conduct additional experiments on three complementary settings: (i) epigenetic mark prediction with larger genomic foundation models, (ii) unsupervised multi-species representation evaluation, and (iii) genomic sequence-to-function and long-range benchmarks. These experiments further validate the extensibility of the Benchmarker module and provide a broader comparison with recent genomic models and benchmark suites.

GUE Epigenetic Marks with Large Models. We first evaluate larger genomic foundation models on the 10 epigenetic mark prediction tasks from the Genome Understanding Evaluation (GUE) benchmark (Zhou et al., 2024). We compare GenomeOcean-4B and EVO 2-7B using LoRA fine-tuning and report the average Matthews correlation coefficient (MCC) across all 10 tasks. GenomeOcean-4B achieves an average MCC of 60.43, while EVO 2-7B achieves 61.77. These results show that GenomeOcean-4B remains competitive with a substantially larger state-of-the-art generative genomic model under the same parameter-efficient fine-tuning protocol.

Unsupervised Multi-species Representation Evaluation. To evaluate whether model embeddings capture biologically meaningful species-level structure without supervised labels, we extract mean-pooled sequence embeddings on the Genomic Benchmarks species classification task (Grešová et al., 2023). We then apply k -means clustering to the embeddings and measure the adjusted Rand index (ARI) against the ground-truth species labels. As shown in Table 7, GenomeOcean-4B achieves the strongest unsupervised clustering performance, nearly doubling the next-best model. This suggests that its learned representations capture richer multi-species biological structure without task-specific supervised training.

Table 7. **Unsupervised representation quality on the Genomic Benchmarks species classification task.** We extract mean-pooled embeddings, apply k -means clustering, and report the adjusted Rand index (ARI) against ground-truth species labels.

Model	DNABERT-2	HyenaDNA-160k	EVO 2-7B	GenomeOcean-4B
ARI	0.167	0.146	0.082	0.321

Evaluation on GUANinE and Long-range Genomic Benchmarks. We further expand the evaluation to include GROVER and two additional benchmark suites: GUANinE v1.0 (Ioannidis et al., 2024) and the Genomics Long-Range Benchmark (LRB) (Trop et al., 2024). GUANinE provides sequence-to-function tasks for genomic AI models, while LRB emphasizes long-range genomic modeling. We evaluate all models using LoRA fine-tuning and report MCC. Table 8 summarizes the results across GUE, Genomic Benchmarks, GUANinE, and LRB.

Table 8. **Additional benchmark results with broader benchmark suites.** We report MCC under LoRA fine-tuning. GUE denotes the Genome Understanding Evaluation benchmark, GB denotes Genomic Benchmarks, GUANinE denotes GUANinE v1.0, and LRB denotes the Genomics Long-Range Benchmark.

Model	GUE	GB	GUANinE	LRB
GROVER	61.38 $\pm$ 0.33	65.42 $\pm$ 0.27	58.15 $\pm$ 0.41	42.73 $\pm$ 0.55
DNABERT-2	65.25 $\pm$ 0.25	71.97 $\pm$ 0.20	62.80 $\pm$ 0.35	45.19 $\pm$ 0.48
HyenaDNA-160k	59.91 $\pm$ 0.22	66.71 $\pm$ 0.18	55.47 $\pm$ 0.39	51.36 $\pm$ 0.62
GenomeOcean-100M	65.52 $\pm$ 0.57	68.26 $\pm$ 0.26	63.14 $\pm$ 0.28	48.92 $\pm$ 0.51

On GUANinE, GenomeOcean-100M obtains the best performance, suggesting strong sequence-to-function adaptation under LoRA fine-tuning. On LRB, HyenaDNA achieves the best result, likely because its long-context architecture is better suited to long-range genomic dependencies. Together, these results show that GENOME-FACTORY can support broader benchmark coverage and more diverse evaluation settings beyond the original GUE and Genomic Benchmarks experiments.

G Usage Examples of GENOME-FACTORY

We provide usage examples of GENOME-FACTORY through two interfaces: the command-line interface (CLI) and the web-based interface (WebUI). We begin with detailed CLI usage examples, including (i) data downloading in Appendix G.1, (ii) data processing in Appendix G.2, (iii) model training in Appendix G.3, (iv) model inference in Appendix G.4, and (v) biological interpretation in Appendix G.5. Then we conclude with the WebUI demonstration in Appendix G.6.

Genome-Factory relies on YAML configuration files to define tasks, with example files available in “*genomeFactory/Examples*”. Users can customize the parameters in these files as long as the required YAML structure is preserved.

G.1 Data Downloading

Users can download data from NCBI. It supports downloading from a config file or an interface.

Using a Config File. Specify download parameters in a YAML file. It supports both species-based and link-based downloads.

- Download by species.

```
genomefactory -cli download genomeFactory/Examples/download_by_species.yaml
```

- Download by Link.

```
genomefactory -cli download genomeFactory/Examples/download_by_link.yaml
```

Using the Interactive Interface. Run the command without a config file and follow the prompts to specify your download criteria (this supports both species-based and link-based downloads).

```
genomefactory -cli download
```

For both two methods, the list of species and their taxonomy IDs used for downloads is stored in “*genomeFactory/Data/Download/Datasets_species_taxonomy_dict.json*”. Users can extend it by adding new species–taxonomy ID pairs to enable downloads for additional species.

G.2 Data Processing

GENOME-FACTORY provides tools to prepare data for model fine-tuning. This includes (i) processing data downloaded from NCBI (Appendix G.2.1), (ii) formatting users’ own custom datasets (Appendix G.2.2), and (iii) processing specialized genomic data (Appendix G.2.3).

G.2.1 PROCESSING NCBI DATA

Follow the steps below to process the data downloaded from NCBI.

- Gather data downloaded into a single folder (steps in Appendix G.1 finish this automatically).
- Run the processing command with a config file.

```
genomefactory -cli process genomeFactory/Examples/process_normal.yaml
```

- The processed data will be ready for model fine-tuning.

G.2.2 PREPARING CUSTOM DATASETS

Follow the steps below to process the custom data.

- Separate the data into three CSV files: “*train.csv*”, “*dev.csv*”, and “*test.csv*”.
- Each CSV file must have two columns: (i) The first column should contain the DNA sequences (e.g., “*sequence*”). (ii) The second column should contain the corresponding labels (e.g., “*label*”). For classification tasks, labels should be integers (e.g., 0, 1, 2). For regression tasks, labels should be continuous numbers.

- Place these three CSV files (“*train.csv*”, “*dev.csv*”, and “*test.csv*”) together in a single folder.
- Specify this folder as the input data directory in your training configuration YAML file.

G.2.3 SPECIALIZED PROCESSING

GENOME-FACTORY provides specialized dataset generation tools for common genomic tasks.

- Promoter region dataset: Generate promoter vs. non-promoter classification data for human, mouse, and zebrafish genomes.

```
genomefactory -cli process genomeFactory/Examples/process_promoter.yaml
```

- Epigenetic mark dataset: Create gene body sequences with H3K36me3 signal classification for human and mouse genomes.

```
genomefactory -cli process genomeFactory/Examples/process_emp.yaml
```

- Enhancer region dataset: Build enhancer vs. non-enhancer classification data for human and mouse genomes.

```
genomefactory -cli process genomeFactory/Examples/process_enhancer.yaml
```

All three datasets feature quality control, configurable train/val/test splits, and output CSV files with “*sequence*”, “*label*” format.

G.3 Model Training

To fine-tune GFMs, GENOME-FACTORY supports two primary task types: classification and regression. Users can specify the desired *task_type* in the training YAML configuration file.

Follow the steps below to fine-tune GFMs using three different methods.

Full-parameter Fine-tuning. The following command applies to all models except for EVO. We do not support the full-parameter fine-tuning for EVO now.

```
genomefactory -cli train genomeFactory/Examples/train_full.yaml
```

Low-rank Adaptation Fine-tuning (LoRA).

- For general models (except for EVO), users can use:

```
genomefactory -cli train genomeFactory/Examples/train_lora.yaml
```

- For the EVO model, users need to use:

```
genomefactory -cli train genomeFactory/Examples/train_evo_lora.yaml
```

For all models, users can specify target modules in the YAML file: (i) “*all*” - targets all linear layers; (ii) “*all_in_and_out_proj*” - targets input/output projection linear layers and the final classification layer; and (iii) “*custom*” - specifies module names by users.

Adapter-based Fine-tuning. The following command applies to all models.

```
genomefactory -cli train genomeFactory/Examples/train_adapter.yaml
```

In “*genomeFactory/Train/workflow/adapter/adapter_model/Adapter.py*”, users can customize the adapter architecture for better performance on specific downstream tasks.

For all three methods, training settings (e.g., batch size, learning rate, and training epochs) can be customized in the respective YAML files.

Benchmarking: After fine-tuning, performance metrics are saved to a JSON file. Users can use these metrics for benchmarking (e.g., comparing the performance of different models or tuning methods).

G.4 Model Inference

Follow the steps below to use genomic models for embedding extraction and generation.

Embedding Extraction. Extract the last hidden state embeddings from sequences.

- For the general case (except for EVO):

```
genomefactory -cli inference genomeFactory/Examples/inference_extract.yaml
```

- For EVO:

```
genomefactory -cli inference genomeFactory/Examples/inference_extract_evo.yaml
```

Generation. Generate new DNA sequences based on prompts. Applicable to generative models.

- For GenomeOcean:

```
genomefactory -cli inference genomeFactory/Examples/inference_generation_genomeocean.yaml
```

- For EVO:

```
genomefactory -cli inference genomeFactory/Examples/inference_generation_evo.yaml
```

G.5 Biological Interpretation

GENOME-FACTORY provides comprehensive tools for understanding and interpreting GFM's through a sparse auto-encoder (SAE) to provide biological insights into model behavior.

Follow the steps below to implement the workflow for interpreting.

Train an SAE Model. Use the following command to train SAE with the DNA sequences.

```
genomefactory -cli sae_train genomeFactory/Examples/sae_train.yaml
```

Configure the following parameters in the YAML file. Users need to specify the DNA sequence file as *data_file*. Use *model_name* to assign a name or checkpoint path for the genomic model. Set *d_model* to the output dimension of the genomic model used to generate the embeddings. Define the SAE bottleneck dimension using *d_hidden*. It controls the number of sparse latent features.

```
data_file: "<YOUR_DNA_SEQUENCE_FILE>"
d_model: "<GENOMIC_MODEL_OUTPUT_DIMENSION>"
d_hidden: "<SAE_HIDDEN_DIMENSION>"
model_name: "<MODEL_NAME_AND_PATH>"
```

Downstream Evaluations with Ridge Regression. Users need to generate hidden embeddings from genomic models and save the results as a CSV file at the path specified by "*i*GENOMIC_MODEL_FEATURE_CSV_PATH*i*" (by Appendix G.4). Then use [CLS] tokens embedding or mean-pooling embedding to do further analysis.

- Use the [CLS] token latent embedding for analysis.

```
genomefactory -cli sae_regression genomeFactory/Examples/sae_regression.yaml
```

Configure the following parameters in the YAML file. Users need to specify the DNA sequence embedding file as *csv_path*. Use *sae_checkpoint_path* to assign the checkpoint path of the SAE.

```
csv_path: "<GENOMIC_MODEL_FEATURE_CSV_PATH>"
sae_checkpoint_path: "<SAE_CHECKPOINT_PATH>"
type: "<first_token>"
```

- Use the mean-pooling latent embedding for analysis.

```
genomefactory -cli sae_regression genomeFactory/Examples/sae_regression.yaml
```

Configure the following parameters in the YAML file.

```
csv_path: "<GENOMIC_MODEL_FEATURE_CSV_PATH>"  
sae_checkpoint_path: "<SAE_CHECKPOINT_PATH>"  
type: "<mean>"
```

G.6 Usage via WebUI

Use the following command to access all GENOME-FACTORY functionalities by a graphical interface.

```
genomefactory -cli webui
```

This command launches a web server. Open the provided URL in users' browsers to use the WebUI.